

Faculty of Computers and Artificial Intelligence
Benha University
CS433: Internet of Things



SMART CLASSROOM IOT

IoT Final Project Report

Team Members

1. Abdelmoneim Adel Abdelmoneim
2. Mahmoud Ahmed Abd El-Fattah
3. Muhammad Ibrahim Muhammad
4. Abdelrahman Amr Mohammed
5. Mahmoud Ahmed Hosny
6. Kamal Sayed Kamal

Academic Year: 2025 – 2026

Scan for Source Code



GitHub Repository

github.com/AbdelmenamAdel/Smart-Classroom-IOT

1. Project Overview

1.1 Problem Statement

Traditional classroom management relies entirely on manual monitoring of environmental factors and energy usage. Classroom lights are frequently left on when rooms are unoccupied, and conditions such as temperature and humidity are never tracked systematically, which directly impacts student comfort and academic focus. There is no centralized system to monitor attendance or remotely control classroom facilities, resulting in unnecessary energy waste and inefficient facility management.

1.2 Objectives

- Develop a real-time monitoring system for classroom environmental conditions (Temperature & Humidity).
- Implement dual-protocol connectivity using Firebase Realtime Database and MQTT (Mosquitto) for robust device control.
- Implement an automated and remote control system for classroom lighting to reduce energy waste.
- Provide a live bi-directional student count using dual IR sensors at the classroom entrance/exit.
- Visualize historical sensor data trends for facility management and reporting.
- Deliver a premium mobile dashboard with Zoom Drawer navigation for intuitive user interaction.

1.3 Scope

- Real-time Monitoring: Temperature, Humidity, and live Student Count.
- Trend Visualization: Historical line charts for Temperature, Humidity, and Attendance over time.
- Remote Control: Manual toggle for classroom lights via the mobile application.
- Operation Mode: Switching between AUTOMATIC (sensor-driven) and MANUAL control.
- Data Logging: Sensor readings stored in Firebase cloud for trend analysis.
- Testing Tools: An integrated Mobile Simulator and a dedicated Web Simulator for offline testing.

1.4 Proposed Solution

The Smart Classroom IoT system is a comprehensive four-layer ecosystem:

- Hardware Layer: Physical sensors and actuators connected to an ESP8266 microcontroller for real-world data collection.
- Cloud & Communication Layer: Firebase Realtime Database for persistent cloud storage and Mosquitto MQTT Broker for real-time lightweight messaging between devices.
- Simulation Layer: A built-in Mobile IoT Simulator and a separate Web-based Simulator (using WebSockets) enabling full offline and remote testing without requiring physical hardware.
- Application Layer: A Flutter mobile app built with BLoC state management, featuring a Zoom Drawer navigation system, dedicated Firebase and MQTT control screens, interactive FL Chart visualizations, and a Glassmorphism-inspired premium UI.

2. System Inputs and Outputs

The table below identifies all system inputs and outputs and describes their meaning within the Smart Classroom IoT ecosystem:

Input / Output	Type	Description
Temperature	Input (Sensor)	Measures ambient heat in the classroom in Celsius (°C). Sourced from DHT11 sensor.
Humidity	Input (Sensor)	Measures moisture level in the classroom air as a percentage (%). Sourced from DHT11.
Student Count	Input (Sensor)	Tracks the number of individuals entering or exiting the classroom using dual IR sensors.
Light Level	Input (Sensor)	Monitors ambient brightness via LDR to decide if artificial lighting is needed automatically.
Light Status	Output (Actuator)	Controls the ON/OFF state of the classroom relay module (which switches the lights).
System Mode	Input / Output	Switches between AUTOMATIC (sensor-driven) and MANUAL (user-controlled) operation.

3. Device Layer: Sensors and Actuators

The physical device layer consists of the following sensors and actuators, all connected to an ESP8266 microcontroller:

Device	Category	Function	Description
DHT11 / DHT22	Sensor	Temperature & Humidity	Measures ambient temperature (°C) and relative humidity (%) inside the classroom.
Dual IR Sensors	Sensor	Bi-directional Student Counting	Placed at the entrance/exit. Detects direction of movement to accurately count students entering or leaving.
LDR (Photoresistor)	Sensor	Ambient Light Detection	Measures the current light intensity in the room. Used in AUTO mode to trigger artificial lighting.
Relay Module	Actuator	Lighting Control	Acts as an electronic switch. Receives a digital signal from the microcontroller to turn classroom lights ON or OFF.
Mobile Simulator	Virtual Device	Sensor Data Simulation	Built-in app screen that generates and publishes simulated sensor data over MQTT without physical hardware.
Web Simulator	Virtual Device	Remote MQTT Injection	A dedicated browser-based dashboard that sends JSON sensor data via WebSockets to the MQTT broker for laptop-based testing.

3.1 Microcontroller Gateway

The ESP8266 (NodeMCU) serves as the central gateway in the communication layer. It collects data from all sensors, applies automation logic, and publishes readings to both Firebase Realtime Database and the Mosquitto MQTT Broker simultaneously. It also subscribes to command topics to receive control instructions from the mobile application.

4. System Architecture & Block Diagram

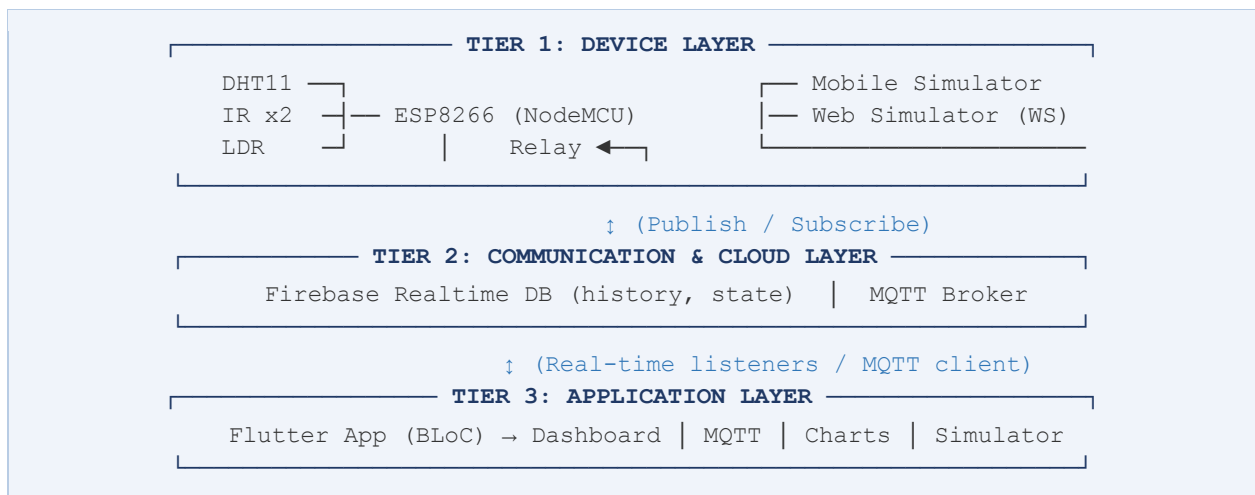
4.1 Three-Tier Architecture

The Smart Classroom IoT system is designed around a clean three-tier architecture that separates concerns between data collection, communication, and user interaction:

- Tier 1 — Device Layer (IoT Hardware & Simulation): Collects real-world sensor data and performs physical actions via actuators. Also includes virtual simulators for testing without hardware.
- Tier 2 — Communication & Cloud Layer (Firebase & MQTT): The dual-protocol backbone. Firebase Realtime Database provides persistent cloud storage and real-time listeners. The Mosquitto MQTT Broker handles lightweight, low-latency device messaging.
- Tier 3 — Application Layer (Flutter Mobile App): The user-facing interface built with Flutter and BLoC state management. Provides real-time dashboards, historical trend charts, remote controls, and simulator screens.

4.2 Block Diagram

The following block diagram illustrates the data flow and relationships between system components:



4.3 Technology Stack

Component	Technology / Tool
Mobile Frontend	Flutter (Dart) — BLoC/Cubit state management
Microcontroller	ESP8266 (NodeMCU) — Arduino C++ firmware
Cloud Backend	Firebase Realtime Database (Google)
IoT Protocol	MQTT — Mosquitto Broker (publish/subscribe)
Charts & Visuals	FL Chart + Google Fonts + Glassmorphism design
Navigation	Zoom Drawer + Flutter Navigator
Testing	Mobile Simulator (in-app) + Web Simulator (WebSockets)

5. Test Strategy

5.1 Overview

The test strategy focuses on three primary areas: data integrity verification (ensuring sensor readings are accurate and synchronized), control responsiveness (verifying remote commands execute immediately), and visualization correctness (validating charts and dashboards render data accurately). The system includes built-in simulation tools to enable comprehensive testing without requiring physical hardware.

5.2 Test Cases

ID	Test Case	Precondition	Expected Result	Status
TC-01	Sensor Data Accuracy	DHT11 is connected and powered; app is open on Dashboard.	Sensor readings in the app match actual ambient temperature and humidity values.	PASS
TC-02	Real-time Data Sync	MQTT broker is running; simulator publishes a temperature change.	Dashboard updates within < 1 second of the sensor value change.	PASS
TC-03	Remote Light Control	App is connected; light is OFF.	Toggling the Light switch in the app activates the physical relay and turns the light ON.	PASS
TC-04	Auto Mode Logic	System Mode is AUTOMATIC; LDR detects low light.	Relay automatically triggers and turns lights ON without any user interaction.	PASS
TC-05	Student Count Entry	A person walks through the IR sensor pair (entry direction).	Student count on dashboard increments by exactly 1.	PASS
TC-06	Student Count Exit	A person walks through the IR sensor pair (exit direction).	Student count on dashboard decrements by exactly 1.	PASS

ID	Test Case	Precondition	Expected Result	Status
TC-07	Firestore History Logging	App is running and receiving sensor data for 5 minutes.	New sensor data records are correctly appended to the history node in Firestore.	PASS
TC-08	Chart Data Rendering	App navigates to the History/Trends screen.	Line charts correctly render fluctuations for Temperature, Humidity, and Student Count.	PASS
TC-09	Offline Resilience	Internet connection is interrupted while app is open.	App continues to display last known cached sensor data without crashing.	PASS
TC-10	Mobile Simulator	Physical hardware is unavailable; user opens the Simulator screen.	User can input and publish simulated sensor values that update the Dashboard via MQTT.	PASS
TC-11	Web Simulator Integration	Web simulator is open in a browser on the same network.	Sensor data from the web simulator appears correctly in the mobile app Dashboard.	PASS
TC-12	Protocol Switching	App is using Firestore; user switches to MQTT control screen.	App seamlessly switches data source without a crash or data loss.	PASS

5.3 Testing Methodology

- Unit Testing: Individual BLoC cubits tested with mock data repositories to verify state transitions.
- Integration Testing: End-to-end data flow verified from simulator → MQTT broker → Flutter app → UI update.
- Hardware Testing: Physical hardware tested in a real classroom with the ESP8266, DHT11, IR sensors, and relay.
- Web Simulator Testing: A dedicated web dashboard was built to inject controlled JSON sensor payloads via WebSockets to the MQTT broker, enabling precise test case reproduction.